

Desenvolvimento Mobile - React Native T1

Projeto Avaliativo - Módulo 1 - Semana 13

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. WIREFRAME DE REFERÊNCIA	2
4. RESULTADOS ESPERADOS (ENTREGA)	3
5. REQUISITOS DAS TAREFAS	4
5.1. PASSO A PASSO SUGERIDO	4
5.2. ORGANIZAÇÃO DO PROJETO	5
5.3. REQUISITOS FUNCIONAIS (RF)	5
5.4. REQUISITOS TÉCNICOS (O QUE DEMONSTRAR DO SEMESTRE)	10
5.5 ORGANIZAÇÃO KANBAN	11
5.6 VERSIONAMENTO GIT/GITHUB	12
5.7. DOCUMENTAÇÃO NO README.MD	12
5.8. GRAVAÇÃO DE VÍDEO	12
6. CRITÉRIOS DE AVALIAÇÃO	13
7. CHECKLIST FINAL DE ENTREGA	15
8. IDEIAS PARA IR ALÉM (OPCIONAL)	16

1. CONTEXTUALIZAÇÃO

Há algumas semanas, você construiu o CineMatch JS: um motor de recomendação que conversava com a pessoa usuária pelo terminal, calculava compatibilidade com um catálogo fictício e devolvia uma recomendação. Funcionou — mas só rodava no seu computador, num terminal preto, e só quem sabia digitar `node cinematch.js` conseguia usar.

Desde então você aprendeu a construir interfaces reais: estruturar páginas com HTML semântico, estilizar com CSS (Flexbox, especificidade, responsividade), capturar dados com formulários, manipular o DOM, reagir a eventos, consumir APIs externas com `fetch`, guardar dados no navegador com `localStorage`, e organizar código em módulos com `Node.js`.

Este projeto final existe pra unir as duas pontas: pegar a lógica que você já validou no terminal e dar a ela uma interface real — um site que qualquer pessoa abre no navegador (inclusive no celular), preenche um formulário, e recebe recomendações de séries reais, buscadas em tempo real de uma API pública.

2. DESAFIO

A PlayNow gostou tanto do protótipo de terminal que quer testá-lo com usuários reais, só que ninguém do time de Produto sabe abrir um terminal. Você foi chamado(a) de volta para transformar o motor de recomendação em uma página web funcional, com formulário, catálogo real de séries e resultado visual.

Você deverá desenvolver um projeto em HTML, CSS e JavaScript chamado:

CineMatch Web: Recomendação de Séries em Tempo Real

A aplicação deverá:

- coletar o perfil da pessoa usuária (nome, idade, gêneros favoritos) por um formulário HTML, não mais pelo terminal;
- guardar esse perfil no navegador com localStorage, para não pedir tudo de novo a cada visita;
- buscar um catálogo real de séries em uma API pública, via fetch;
- filtrar, ordenar e selecionar esse catálogo com métodos de array;
- reaproveitar (adaptando) a lógica de compatibilidade, classes e herança do projeto anterior;
- renderizar tudo dinamicamente na tela, com cards estilizados e responsivos;
- funcionar bem tanto no desktop quanto no celular.

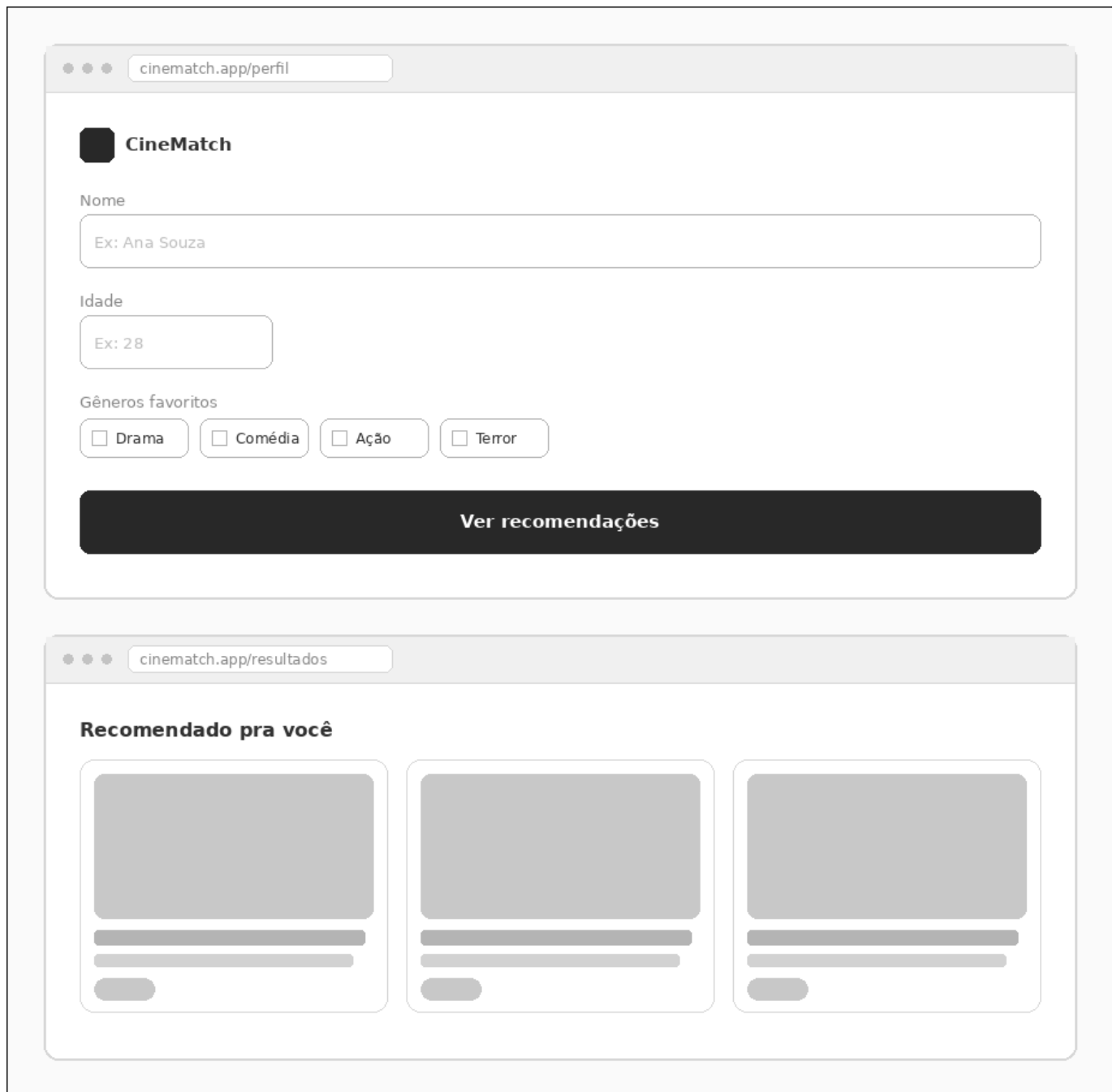
Ao construir a aplicação, você coloca em prática os aprendizados em:

- Lógica e JavaScript (S02–S06): tipos, condicionais, operadores, laços, funções, arrow functions, arrays e seus métodos, objetos, POO (classes, herança, this), callbacks, closures, Promises e async/await, o motor do Cinematch.
- HTML e CSS (S07/S10): marcação, CSS externo, Flexbox, box model, e responsividade mobile-first (a página funciona em qualquer tela).
- DOM e eventos (S08/S09): capturar formulário, validar entradas, reagir a eventos e desenhar a tela com JavaScript (createElement/classList).
- Semântica, acessibilidade e SEO (S11): tags que significam algo, label/for, alt, foco visível, aria-label, title/meta description.
- Persistência e rede (S11): localStorage para lembrar do usuário e fetch + async/await para carregar as séries, tratando os três estados (carregando/vazio/erro).
- Organização e ferramentas (S12): dividir o JavaScript em módulos ES (import/export), usar o ecossistema npm como ferramenta de apoio (opcional).
- Processo profissional: Git/GitHub (branches, commits descritivos), Trello (Kanban) e README, tratar suas alterações como algo que impacta o projeto inteiro.

3. WIREFRAME DE REFERÊNCIA

Antes de programar, vale a pena olhar para onde você quer chegar. O wireframe abaixo mostra dois estados da aplicação: o formulário de perfil e os resultados com os cards de recomendação. Isso não quer dizer que precisam ser duas páginas — pode ser uma única página, em que os cards aparecem no lugar do formulário depois do envio, ou duas

telas/rotas separadas. A escolha de como estruturar isso é da squad; o que importa é a ideia geral de layout (formulário simples primeiro, cards organizados em grade depois).



4. RESULTADOS ESPERADOS (ENTREGA)

Ao final do projeto, o estudante ou squad (grupo de até 03 pessoas) deverá entregar:

- Um repositório no GitHub público (para acesso via link) com a aplicação rodando (HTML + CSS + JS).
- Um arquivo `index.html`, um `style.css` e os arquivos JavaScript.
- Um arquivo `README.md` explicando o projeto.
- Um quadro Kanban com as tarefas realizadas.
- Um histórico de commits no GitHub.
- Um vídeo de até 7 minutos demonstrando o funcionamento.

- Os links submetidos no AVA.

Links obrigatórios. O(a) estudante deverá enviar no AVA:

- link do repositório público do projeto no GitHub;
- link de acesso ao quadro Kanban utilizado no projeto (Trello ou similares). Acessível via link;
- link de acesso ao vídeo apresentação do projeto (ser possível de visualizar via link). Via Google Drive ou YouTube como vídeo "não listado";

• **Prazo de entrega: 05/10/26, segunda, até às 22h**

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. Não esqueça de submeter todos links no AVA. Os links não submetidos terão penalidade na nota.
3. Projetos com plágio (cópia da internet ou de colegas) recebem nota 0. Você pode consultar materiais, documentação e exemplos e até pedir ajuda auxílio à IA, desde que adapte ao desafio e saiba explicar cada linha com segurança.

5. REQUISITOS DAS TAREFAS

5.1. PASSO A PASSO SUGERIDO

Se não souber por onde começar, siga esta ordem. Ela evita o erro mais comum (tentar montar tudo de uma vez) e te leva a testar cada pedacinho antes de seguir pro próximo — a referência entre parênteses aponta pro requisito funcional correspondente.

- Crie os arquivos vazios (index.html, style.css, script.js, ui.js e modelo.js, já dentro de uma pasta com git iniciado.)
- RF01, RF02 Monte o HTML da tela de formulário (sem estilo ainda — só a estrutura semântica e os campos de nome, idade e gêneros.)
- RF09 Estilize essa primeira tela (cores, espaçamento e já pensando em Flexbox. Use o wireframe da seção 3 como referência.)
- RF02 Capture os dados do formulário (no evento submit, monte o objeto usuario e confirme com um console.log.)
- RF03 Salve o perfil no localStorage (e teste recarregando a página — o perfil deve continuar lá.)
- RF04 Busque o catálogo com fetch (dentro de um try/catch, e mostre o resultado bruto no console antes de tratar qualquer coisa. Teste também o que acontece se a busca falhar (ex: desligue a internet um instante) — a página não pode travar.)
- RF05 Trate esse catálogo (com filter, sort, slice e map, até chegar num array parecido com o do projeto anterior.)
- RF06 Adapte as classes do projeto anterior (Conteudo e Serie, agora dentro de [modelo.js](#).)
- RF07 Calcule a compatibilidade - ainda só no console, pra validar a lógica antes de desenhar a tela.

- **RF08** Monte e renderize a tela de resultados - gerando os cards dinamicamente a partir dos dados já calculados.
- **RF09** Estilize a tela de resultados - Flexbox e responsividade pra grade de cards.
- **RF10-RF12** Adicione os detalhes finos - callback, closure e a mensagem de carregamento com setTimeout.
- **RF13** Revise SEO e acessibilidade - meta tags, labels dos campos, alt em imagens.
- **RF14** Separe script.js, ui.js e modelo.js com import/export - e sirva o projeto com live-server pra rodar como módulo.
- Teste tudo do zero - limpe o localStorage, recarregue, preencha o formulário de novo e confira se os resultados aparecem certo.
- Feche o projeto - atualize o README, grave o vídeo e envie os links no AVA.

5.2. ORGANIZAÇÃO DO PROJETO

A estrutura mínima do projeto deverá ser:

```
cinematch-web/
├── index.html
├── style.css
├── script.js    (formulário, localStorage, fetch, cálculo, orquestração)
├── ui.js       (renderização dos cards, mensagens de carregamento/erro)
├── modelo.js   (classes Conteudo e Serie, exportadas)
└── README.md
```

Três arquivos JavaScript, cada um com uma responsabilidade clara: script.js comanda o fluxo (formulário, localStorage, busca e cálculo), ui.js cuida de tudo que toca a tela (renderizar os cards, mostrar mensagens de carregamento ou erro), e modelo.js guarda as classes. Isso já cumpre com folga o requisito de módulos ES (import/export) — não é preciso ir além disso. Se sua squad preferir separar mais (por exemplo, um arquivo só pra chamada da API), pode — mas é opção de organização, não exigência.

5.3. REQUISITOS FUNCIONAIS (RF)

RF01 — Estruturar a página com HTML semântico

A página deverá usar elementos semânticos em vez de <div> para tudo: <header>, <main>, <section>, <article> (um por série recomendada) e <footer>. Inclua também <title> e uma <meta name="description">, utilize as og tags também — é o que já foi estudado no bloco de HTML semântico/SEO.

RF02 — Criar o formulário de perfil:

Substitui o prompt-sync do projeto anterior. Exemplo de estrutura:

```
<form id="form-perfil">
  <label for="nome">Nome</label>
```

```

<input type="text" id="nome" required>

<label for="idade">Idade</label>
<input type="number" id="idade" min="1" required>

<fieldset>
  <legend>Gêneros favoritos</legend>
    <label><input type="checkbox" name="genero" value="Drama">
Drama</label>
    <label><input type="checkbox" name="genero" value="Comedy">
Comédia</label>
    <label><input type="checkbox" name="genero" value="Action">
Ação</label>
    <!-- adicione outros gêneros -->
  </fieldset>

  <button type="submit">Ver recomendações</button>
</form>

```

A captura deve acontecer no evento submit, com preventDefault(), montando o mesmo tipo de objeto usuario do projeto anterior (nome, idade, generosFavoritos). Lembre-se de colocar tratativa de erro para o formulário. E exibir mensagem/feedback visual para o usuário

RF03 — Persistir o perfil com localStorage

Ao enviar o formulário, salve o perfil no navegador. Na próxima visita, se já existir um perfil salvo, pule o formulário e vá direto para o catálogo (com um botão "Trocar perfil" para o caso de querer recomeçar).

```

localStorage.setItem('cinematchPerfil', JSON.stringify(usuario));

const perfilSalvo = localStorage.getItem('cinematchPerfil');
if (perfilSalvo) {
  const usuario = JSON.parse(perfilSalvo);
  // pula o formulário e mostra o catálogo direto
}

```

RF04 — Buscar o catálogo real via fetch

Em vez do array fictício do mini-projeto anterior, o catálogo agora vem de uma API real e pública, sem necessidade de chave: a [TVMaze API](#).

Por que o tratamento de erro importa aqui

O array fictício do CineMatch JS nunca falhava — ele estava sempre ali, no código. Uma chamada de rede é diferente: a internet pode cair, a API pode estar fora do ar, a resposta pode vir vazia ou num formato inesperado. Sem tratar isso, a página trava ou fica em branco sem explicação nenhuma pra pessoa usuária. Tratar erro não é um extra — é parte do requisito de consumir uma API real.

```
async function buscarCatalogo() {  
  const resposta = await fetch('https://api.tvmaze.com/shows?page=0');  
  ...  
}
```

Use `async/await` (mantendo o requisito do projeto anterior, agora com dado real), sempre dentro de um `try/catch`. Não precisa ser sofisticado — o mínimo é: se a busca falhar, mostrar uma mensagem de erro na tela em vez de deixar a página quebrada ou o formulário sem resposta nenhuma.

RF05 — Tratar o catálogo com métodos de array

Nem toda série retornada pela API tem gênero ou nota preenchidos. Trate os dados antes de usar:

```
const catalogo = dados  
  .filter(serie => serie.genres.length > 0 && serie.rating.average)  
  .sort((a, b) => b.rating.average - a.rating.average)  
  .slice(0, 8)  
  .map(serie => ({  
    id: serie.id,  
    titulo: serie.name,  
    tipo: "Série",  
    generos: serie.genres,  
    duracaoMinutos: serie.runtime,  
  }));
```

Use pelo menos 3 métodos de array entre: `filter`, `sort`, `slice`, `map`, `find`, `every`, `reduce` — igual ao projeto anterior, mas aplicado num dado real.

E se o catálogo chegar vazio (porque o RF04 caiu no `catch`, ou porque o filtro não sobrou nada)? Trate esse caso também — uma mensagem simples como "Não encontramos recomendações agora" é melhor do que renderizar uma grade de cards vazia sem explicação.

RF06 — Reaproveitar e adaptar as classes do projeto anterior

Traga as classes `Conteudo` e `Serie` (com herança e uso de `this`) do `CineMatch JS` para dentro de `modelo.js`. Adapte o construtor, se necessário, para receber os dados já tratados no RF05.

RF07 — Calcular e classificar a compatibilidade

Mantém a mesma regra do projeto anterior (gêneros em comum / total de gêneros do conteúdo × 100) e a mesma classificação por faixa (Alta/Média/Baixa afinidade), usando `if-else`, `switch-case` ou `ternário` — só que agora o resultado é exibido na tela, não no console.

RF08 — Renderizar os resultados no DOM

Gere dinamicamente um `<article>` (card) para cada série recomendada, mostrando título, gêneros em comum, gêneros não explorados, percentual de compatibilidade e classificação. Isso substitui o antigo menu do terminal: aqui, os resultados aparecem direto na tela, sem precisar de opções numeradas. É natural que essa função viva em `ui.js`, já que é responsabilidade de tela, não de lógica.

```
// ui.js
export function renderizarCard(resultado) {
  const card = document.createElement('article');
  card.className = 'card-serie';
  card.innerHTML = `
    <h3>${resultado.titulo}</h3>
    <p>Compatibilidade: ${resultado.percentual}%</p>
    <span class="badge
  ${resultado.classificacao}">${resultado.classificacao}</span>
  `;
  document.querySelector('#resultados').appendChild(card);
}
```

RF09 — Estilizar com Flexbox e responsividade

Os cards devem ficar organizados em Flexbox (linha no desktop, coluna no celular), usando media queries para o breakpoint mobile. Cuide da especificidade do CSS: prefira classes bem nomeadas em vez de `!important` ou seletores de tag muito genéricos.

RF10 — Usar callback

Crie uma função de callback disparada quando o catálogo termina de carregar — por exemplo, uma função `exibirMensagemDeBoasVindas(nome)` chamada após a busca e renderização inicial.

RF11 — Usar closure

Mantenha (ou recrie) um contador por closure — por exemplo, quantas vezes a pessoa recalculou a compatibilidade nesta sessão — e exiba esse número na tela.

RF12 — Usar uma Browser API de tempo

Enquanto o `fetch` busca os dados, mostre uma mensagem de carregamento (ex.: "Buscando as melhores séries pra você..."). Pode usar `setTimeout` para um pequeno atraso proposital na exibição, reforçando o uso de `setTimeout/setInterval` do bloco de Browser APIs.

Bônus opcional: se quiser ir além, dá pra usar a Geolocation API pra uma sugestão contextual (por exemplo, uma saudação ou recomendação diferente dependendo de onde a pessoa está acessando). Não é obrigatório — o setTimeout já cumpre o requisito — mas é uma boa forma de explorar outra Browser API além da vista em aula.

RF13 — SEO básico e acessibilidade

Título e meta description descritivos, <label> associado a cada campo do formulário (via for/id), texto alternativo em qualquer imagem usada, e contraste de cores adequado.

RF14 — Organizar o código em módulos ES (import/export)

Separe modelo.js e ui.js de script.js usando import/export, carregado a partir de index.html com <script type="module" src="script.js"></script>. Três arquivos já cumprem o requisito com folga — não precisa fatiar mais que isso.

```
// modelo.js
export class Conteudo { /* ... */ }
export class Serie extends Conteudo { /* ... */ }

// ui.js
export function renderizarCard(resultado) { /* ... */ }
export function exibirMensagemDeErro(texto) { /* ... */ }

// script.js
import { Conteudo, Serie } from './modelo.js';
import { renderizarCard, exibirMensagemDeErro } from './ui.js';
```

No README.md, explique em poucas linhas a diferença entre CommonJS (require/module.exports, usado no CineMatch JS original) e ESM (import/export, usado aqui) — é um dos conteúdos da semana de [Node.js](#).

RF15 — Servir o projeto com um pacote via npm

Instale e use um pacote simples para servir a pasta localmente. Reforça gerenciador de pacotes e instalação global vs. local, vistos na aula de introdução ao [Node.js](#).

Exemplo de script do package.json

```
{
  "name": "cinematch-web",
  "scripts": {
    "start": "live-server"
  },
  "devDependencies": {
    "live-server": "^1.2.2"
  }
}
```

}

5.4. REQUISITOS TÉCNICOS (O QUE DEMONSTRAR DO SEMESTRE)

Conteúdo (semana)	Obrigatório?	Como demonstrar
Tipos, operadores, condicionais (if/else, switch, ternário) (S02–S03)	Sim	Cálculo de compatibilidade e classificação (RF07)
Laços (for/while) ou métodos de array que iteram (S03–S04)	Sim	Os métodos de array do RF05 já contam; um laço explícito (ex.: montar a lista de gêneros não explorados) é bem-vindo
Funções e arrow functions (S03)	Sim	Regras de compatibilidade; callbacks dos eventos do formulário e dos cards
Arrays + métodos map/filter/find/every/reduce (S04)	Sim (≥3)	Tratar o catálogo e comparar gêneros (RF05)
Objetos (chaves/valores) (S04)	Sim	Perfil da pessoa usuária e cada série do catálogo
POO: classe, construtor, atributos, método, this, herança (S05)	Sim	Classe Conteúdo + Serie (RF06)
Callback e closure (S06)	Sim	Mensagem de boas-vindas (callback) ou Mensagem final; contador de recálculos (closure)
Promises / async-await (S06)	Sim	No fetch (RF04) — agora real, não simulado
HTML semântico (S07/S11)	Sim	Landmarks (header/main/section/article/footer), um h1, hierarquia (RF01)
CSS externo, box model, Flexbox (S07/S10)	Sim	Layout dos cards e do formulário (RF09)
Responsividade mobile-first (media queries, unidades fluidas) (S10)	Sim	Funciona do celular ao desktop (RF09)
Especificidade/cascata (S10)	Sim	Estilos sem conflito; explicar no README se usar !important
Bootstrap / Font Awesome via CDN (S10)	Opcional	Se usar, só via CDN e sem npm install

DOM + eventos: addEventListener, createElement, classList, preventDefault (S08/S09)	Sim	Formulário + render dos cards (RF02, RF08)
Validação de formulário (S09)	Sim	Campos obrigatórios + erro acessível
Acessibilidade: label/for, alt, aria-label, foco, lang (S11)	Sim	Página navegável por teclado
SEO on-page: title, meta description (S11)	Sim	No <head> (RF01, RF13)
fetch + 3 estados (carregando/vazio/erro), try/catch, response.ok (S11)	Sim	Carregar o catálogo — carregando (RF12), vazio e erro (RF04, RF05)
LocalStorage + JSON.stringify/parse + tratar null (S11)	Sim	Lembrar o perfil da pessoa usuária (RF03)
Geolocation / outra Browser API (S11)	Opcional	Ex.: Geolocation para uma sugestão contextual, além do setTimeout do RF12 (Possível melhoria futura para ver de onde são os usuários que mais acessam)
Módulos ES import/export (S12)	Sim	script.js / ui.js / modelo.js separados (RF14)
npm / package.json / script de dev (S12)	Sim	Servidor local de desenvolvimento (ex.: live-server, RF15)
Git: branches, commits descritivos, GitHub (S06+)	Sim	Histórico do repositório
Kanban (Trello)	Sim	Quadro do projeto
var/let/const e escopo de bloco	Sim	Priorizar const/let; explicar as escolhas (e onde o escopo importou) no README

Fora do escopo do Módulo 01 (não usar): React/Next/React Native ou qualquer framework JS (Vue/Angular), TypeScript, bundlers/build (Webpack/Vite/Babel), CSS Grid como sistema de layout (foi só menção, use Flexbox), Sass/CSS-in-JS, back-end/servidor/banco de dados, fetch com POST/PUT/DELETE que grave em servidor, jQuery/axios e bibliotecas não vistas. Tudo isso chega no Módulo 02 — aqui o mérito é fazer com o que aprendemos.

5.5 ORGANIZAÇÃO KANBAN

- Colunas obrigatórias: Backlog, A Fazer, Em Andamento, Concluído.

- Pode usar: Trello, GitHub Projects, Notion, quadro simples no README, planilha ou imagem do quadro.
- As tarefas do passo a passo sugerido (5.1) já servem de base pronta pras colunas do Kanban — não precisa reinventar a lista.

5.6 VERSIONAMENTO GIT/GITHUB

- Branches mínimas (individual): main, develop, feature/cinematch-web.
- Branches mínimas (squad): main, develop, feature/interface (formulário + estilos) e feature/logica (API + classes + compatibilidade).
- Não é preciso criar uma branch pra cada RF — o objetivo é mostrar que você sabe separar trabalho em uma branch de feature e trazer de volta pra develop, não multiplicar branches. **Todo o código de todos deve estar na branch main ao final do projeto**

Commits mínimos: 5 (individual) / 8 (squad).

Exemplos de commits:

- feat: cria estrutura semântica do index.html
- feat: adiciona formulário de perfil com persistência em localStorage
- feat: busca catálogo real via fetch na TVMaze API
- feat: adapta classes Conteudo e Serie do projeto anterior
- feat: renderiza cards de recomendação no DOM
- style: estiliza cards com flexbox e responsividade
- docs: atualiza readme com instruções de execução

5.7. DOCUMENTAÇÃO NO README.MD

Crie um arquivo readme.md no repositório do seu projeto no GitHub, para documentar a sua solução, bem como demonstrar as técnicas e linguagens utilizadas, além do escopo do projeto e como o usuário pode executar o seu sistema.

Algumas dicas interessantes para utilizar na criação do seu portfólio são:

- Criar um nome para o seu software;
- Descrever qual o problema ele resolve;
- Descrever quais técnicas e tecnologias utilizadas. Aqui você também pode inserir alguma imagem ou diagrama para melhor entendimento;
- Descrever como executar;
- Descrever quais melhorias podem ser aplicadas;
- Entre outras coisas.

5.8. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento deste sistema você deverá gravar um vídeo, com tempo **máximo de 7 minutos**, abordando os seguintes questionamentos:

- Qual o objetivo do sistema? E demonstração de funcionamento.
- O que deve ser realizado para executar o sistema?
- Como você organizou as tarefas antes de começar a desenvolver?
- Quais as *branches* você criou e quais os objetivos para cada uma?
- Você acha que faltou algo no seu código que você poderia melhorar?

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto e esteja em um local com boa iluminação. Para realizar a entrega do vídeo, coloque em uma pasta do Google Drive em modo leitor para qualquer pessoa com o link, e compartilhe o mesmo na submissão do projeto no AVA. Uma dica interessante é você inserir o vídeo no `readme.md` do seu projeto no repositório do GitHub.

6. CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 60% sobre a avaliação do módulo.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Apresentação do Projeto				
Nº	Critério de Avaliação	0	1,00	1,50
1	Realizou a gravação de um vídeo?	Não foi realizada a gravação do vídeo.	Gravou o vídeo e abordou parte dos tópicos listados no item 5.8.	Gravou o vídeo e abordou todos os tópicos listados no item 5.8.

Uso do GitHub e Readme.md				
Nº	Critério de Avaliação	0	0,25	1,00
2	Versionamento com branches e commits	O repositório do projeto não apresenta branches e commits.	O repositório do projeto apresenta parte das branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.	O repositório do projeto apresenta branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.
3	Organização dos arquivos no repositório	O repositório do projeto não apresenta os arquivos (estrutura de pastas e README.md) estruturados conforme as instruções.	O repositório do projeto apresenta parte dos arquivos (estrutura de pastas e README.md) estruturados conforme as instruções.	O repositório do projeto apresenta os arquivos (estrutura de pastas e README.md) estruturados conforme as instruções.

Desenvolvimento da Aplicação				
Nº	Critério de Avaliação	0	0,25	0,50
4	Modelagem do perfil da pessoa usuária e catálogo de séries	Não modelou o perfil nem o catálogo.	Modelou apenas um dos dois, ou em formato incompleto.	Modelou o perfil da pessoa usuária em objeto e o catálogo de séries estruturado como uma coleção (array) de objetos.
Nº	Critério de Avaliação	0	0,50	1,00
5	Compatibilidade entre perfil e catálogo	Não calculou a compatibilidade entre o perfil e o catálogo.	Calculou a compatibilidade com erros, ou deixou de classificar as séries ou de apontar os gêneros não explorados.	Implementou o cálculo da compatibilidade (gêneros em comum e não explorados) e a classificação das séries em Alta/Média/Baixa afinidade.
Nº	Critério de Avaliação	0	0,25	0,50
6	Utilização de métodos	Não utilizou métodos de array, ou utilizou apenas um.	Utilizou dois métodos de array.	Utilizou pelo menos três métodos de arrays (map, filter, find, every, reduce) para realizar a filtragem e a transformação de listas.
7	Utilização de métodos de classe	Não criou nenhuma classe.	Criou a classe, mas não incluiu a herança, ou criou a herança sem novas funcionalidades.	Criou a classe com this e herança justificada, em que a subclasse acrescenta ou sobrescreve comportamento.
8	Emprego de callback e closure	Não empregou callback nem closure.	Empregou apenas um dos dois.	Empregou closures para a preservação do estado de variáveis locais e funções de callback para o controle do fluxo de execução.
9	Estrutura da página HTML	Estruturou a página sem semântica ("div-soup") e sem acessibilidade.	Utilizou tags semânticas, mas falhou em itens de acessibilidade ou de SEO.	Utilizou landmarks e um único h1, associou rótulos aos campos, usou alt e foco visível, e cuidou do SEO on-page na estrutura da página HTML
10	Responsividade	A aplicação quebrou no celular.	Deixou a aplicação parcialmente responsiva, com poucos breakpoints.	Aplicou responsividade com Flexbox, unidades fluidas e <i>media queries</i> , funcionando em dispositivos móveis e desktop.

11	Formulário e validação	Não construiu o formulário, ou não tratou os eventos.	Construiu o formulário, mas sem validação ou sem preventDefault.	Construiu o formulário com addEventListener, preventDefault e validação com mensagem de erro acessível.
12	Conteúdo dinâmico	Deixou os cards fixos no HTML.	Gerou os cards parcialmente, misturando HTML fixo e JavaScript.	Gerou os cards inteiramente por JavaScript, a partir dos dados.
13	Consumo de dados	Não utilizou fetch.	Utilizou fetch, mas não tratou os três estados.	Utilizou fetch com try/catch e response.ok, tratando os estados de carregando, vazio e erro.
14	LocalStorage	Não persistiu nenhum dado.	Persistiu os dados, mas quebrou no null da primeira visita ou não usou JSON.	Persistiu os dados com JSON.stringify/parse e tratou o null da primeira visita.
15	Módulos ES	Manteve todo o código em um único arquivo, sem evidência de depuração.	Organizou em módulos ES, mas utilizou uma versão menos atual.	Separou o código em módulos ES.

7. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira:

- ☐ Criei o repositório público no GitHub
- ☐ Criei o index.html com HTML semântico
- ☐ Criei o style.css com Flexbox e responsividade
- ☐ Criei o formulário de perfil (nome, idade, gêneros)
- ☐ Implementei localStorage para persistir o perfil
- ☐ Busquei o catálogo real via fetch na TVMaze API
- ☐ Tratei erros da chamada à API (try/catch e mensagem amigável se falhar ou vier vazio)
- ☐ Tratei o catálogo com pelo menos 3 métodos de array
- ☐ Adapte as classes Conteudo e Serie (herança e this)
- ☐ Calculei e classifiquei a compatibilidade
- ☐ Renderizei os resultados dinamicamente no DOM
- ☐ Usei callback
- ☐ Usei closure
- ☐ Usei uma Browser API de tempo (setTimeout/SetInterval/Geolocation)
- ☐ Separei script.js, ui.js e modelo.js com import/export
- ☐ Servi o projeto com um pacote local (ex.: live-server)
- ☐ Cuidei de SEO básico e acessibilidade
- ☐ Testei em desktop e celular
- ☐ Criei o README.md
- ☐ Criei o quadro Kanban
- ☐ Fiz commits e usei ao menos uma branch de feature no GitHub
- ☐ Gravei o vídeo de até 7 minutos

- [] Coloquei o vídeo com permissão correta para ver via link
- [] Enviei os links no AVA

8. IDEIAS PARA IR ALÉM (OPCIONAL)

Nada aqui é requisito de nota — é pra quem terminou os RFs e quer deixar o projeto mais redondo pro portfólio, ou simplesmente curtiu o desafio e quer continuar mexendo. Duas frentes possíveis: resgatar funcionalidades que existiam no CineMatch JS (terminal) e ficaram pra trás na versão web, ou ir além do que qualquer uma das duas versões já tinha.

Melhorias que vão além dos dois projetos

- Filtro por gênero na tela de resultados, pra refinar sem preencher o formulário de novo.
- Ordenar os cards por compatibilidade, nome ou avaliação (reaproveitando sort).
- Buscar mais de uma página da TVMaze API (?page=1, ?page=2...) pra ampliar o catálogo disponível.
- Publicar o projeto de verdade — GitHub Pages, Netlify ou Vercel, pra ter um link público de verdade pra colocar no portfólio.
- Modo escuro — um toggle simples de tema, reforçando manipulação de classes via classList.
- Combinar com uma API de filmes — voltar a ter "filmes e séries" como no mini-projeto original, combinando a TVMaze com outra fonte de dados